

The Maharaja Sayajirao University of Baroda
Faculty of Technology & Engineering

A PROJECT REPORT

on

"Intelligent File Deduplication Engine"

Submitted by

Prajapati Rutvij (386200)

Thanki Parth (386230)

Vasava Rohan (386238)

Guided by

Dr. Mamta Padole

Department of Computer Science and Engineering
2026-27

ABSTRACT

In modern personal and enterprise computing environments, uncontrolled accumulation of redundant data poses a persistent challenge to filesystem performance, storage economics, and operational clarity. Users frequently duplicate multimedia, documents, and disk images across directories, often renaming or relocating copies without altering their internal content. Conventional file management utilities that rely solely on superficial metadata (such as filenames, timestamps, or basic extension filters) suffer from high rates of false positives and false negatives, risking accidental deletion of distinct files or failing to detect identical content saved under altered aliases.

The Intelligent File DeDuplication Engine (IFDE) is a high-performance desktop application engineered using Java 21 and JavaFX under strict Object-Oriented Programming (OOP) paradigms. IFDE implements a robust Model-View-Controller (MVC) architecture that isolates compute-intensive filesystem operations from the graphical interface. The engine operates through a coordinated Two-Pass Traversal algorithm: the first pass catalogs filesystem objects to establish an exact denominator for progressive feedback, while the second pass streams file content through Java NIO FileChannels into a 64KB Direct ByteBuffer to generate cryptographic SHA-256 digests via MessageDigest. This chunk-based architecture prevents Out-Of-Memory (OOM) failures when processing multi-gigabyte media, maintaining a deterministic heap footprint under 100MB.

Thread-safe synchronization and custom event-driven listeners (ScanListener) ensure that the graphical interface remains fully responsive throughout sustained I/O bursts. Detected duplicates are indexed in a hash-bucketed data store, rendered in a hierarchical JavaFX TreeTableView, and evaluated using an algebraic storage reclamation model. IFDE features automated duplicate selection, confirmation-guarded batch deletion, and resilient exception handling for recursive symbolic link traps and locked filesystem nodes. The solution is compiled via Apache Maven into a standalone, modular distribution, demonstrating practical mastery of core Java concurrency, stream I/O, and modern desktop engineering.

TABLE OF CONTENTS

Chapter No.	Title / Topic	Page No.
—	Cover Page (Institutional & Candidate Credentials)	1
—	Abstract	2
—	Table of Contents	3
Chapter 1	Introduction & Problem Statement	4
Chapter 2	Project Objectives & Scope	5
Chapter 3	System Requirements & Feasibility Analysis	6
Chapter 4	System Architecture & Design	7
Chapter 5	Object-Oriented Programming (OOP) Concepts Implemented	8
Chapter 6	Core Algorithms & Mathematical Formulations	9
Chapter 7	Detailed Module & Class Specification	10
Chapter 8	Edge Cases, Robustness & Defensive Engineering	11
Chapter 9	Output Screenshots & Test Evidence	12–15
Chapter 10	Conclusion, Learning Outcomes & Future Scope	16

CHAPTER 1: INTRODUCTION & PROBLEM STATEMENT

1.1 Project Overview

Personal and enterprise computer systems experience continuous storage degradation driven by the unmanaged proliferation of redundant files. Everyday computing workflows—such as extracting compressed archives, downloading duplicate email attachments, staging software releases, and creating manual folder backups—inevitably scatter identical files across disparate filesystem hierarchies. This redundancy not only depletes expensive Solid State Drive (SSD) storage capacity but also complicates data organization, degrades search indexing performance, and lengthens system backup windows.

The Intelligent File DeDuplication Engine (IFDE) is conceived as a lightweight, cross-platform software utility designed to detect and eliminate duplicate files with mathematical certainty. Developed using Java 21, the system combines high-performance Java New I/O (NIO) stream processing with a modern JavaFX graphical user interface. Rather than relying on unreliable heuristics, IFDE interrogates the intrinsic byte sequences of files using cryptographic SHA-256 hashing, providing users with an automated, reliable platform for storage optimization.

1.2 Problem Statement

Conventional operating system tools and elementary file search utilities typically compare files using basic filesystem attributes, notably filename, creation timestamp, and file extension. This approach introduces severe operational flaws in real-world scenarios:

- **False Positive Vulnerability:** Two completely different files may share an identical filename (e.g., 'document.pdf' or 'presentation.pptx' stored in separate directories). Deleting one based on name matching destroys unique data.
- **False Negative Inefficiency:** When a user renames a duplicate file (e.g., 'IMG_001.jpg' renamed to 'Profile_Pic.jpg'), attribute-based utilities fail to recognize the redundancy, leaving the duplicate undetected.
- **Quadratic Computational Inefficiency:** Elementary programs that compare entire files byte-by-byte perform quadratic $O(N^2)$ comparisons across N files, rendering scans across large media collections prohibitively slow.

Therefore, an intelligent deduplication system must establish identity strictly through cryptographic hash digests with collision probabilities approaching zero, indexed through linear-time O(N) hash tables, while isolating UI responsiveness from high-latency filesystem I/O.

1.3 Purpose and Motivation

The primary motivation for this project is to apply the theoretical principles of Object-Oriented Programming with Java (OOPJ) to build a production-quality systems utility. The project demonstrates the practical integration of multithreading, non-blocking asynchronous event delegation, low-level NIO buffering, exception-resilient recursion, and modular MVC separation. The resulting tool provides everyday users and systems administrators with an open-source, private, and deterministic desktop solution for storage recovery.

CHAPTER 2: PROJECT OBJECTIVES & SCOPE

2.1 Primary Objectives

The development of the Intelligent File DeDuplication Engine was directed by the following concrete engineering objectives:

- **Cryptographic Content Integrity:** Implement SHA-256 cryptographic hashing to ensure absolute content-level duplicate detection, eliminating false positives caused by file renaming.
- **Deterministic Two-Pass Traversal:** Design a decoupled Two-Pass traversal algorithm capable of traversing deeply nested folder trees, counting matching files upfront, and calculating precise 0–100% progress metrics.
- **Non-Blocking Asynchronous UI:** Utilize a dedicated background worker thread alongside JavaFX Platform.runLater() event dispatching, ensuring the graphical dashboard remains fluid and responsive during heavy I/O.
- **Bounded Memory Footprint:** Process large files (including multi-gigabyte disk images and video files) through a 64KB Direct ByteBuffer, guaranteeing constant memory consumption under 100MB RAM.
- **Automated Storage Recovery:** Provide an intelligent Auto-Select algorithm that preserves one master file per duplicate cluster while staging all redundant copies for batch deletion.
- **Defensive Fault Tolerance:** Incorporate defensive handling for symlink cycles (e.g., Wine/Linux virtual drive loops) and unreadable system nodes without abnormal application termination.

2.2 Scope of the Application

The functional scope of IFDE encompasses desktop file deduplication across local and mounted block storage devices:

- **Cross-Platform Local Storage:** Fully supported across standard desktop operating systems including GNU/Linux (Arch, Ubuntu, Fedora), Microsoft Windows 10/11, and Apple macOS.
- **Selective & Global File Filtering:** Capable of filtering target file extensions (e.g., Images: .jpg, .png; Videos: .mp4, .mkv; Documents: .pdf, .docx) or scanning all file formats indiscriminately.
- **Interactive Visual Categorization:** Calculates cumulative wasted disk capacity and groups files into expandable visual clusters displaying file path, size, last modification timestamp, and SHA-256 hash.
- **Self-Contained Deployment:** Packaged via Apache Maven and the Maven Shade Plugin into a self-contained Fat JAR (ifde.jar) requiring zero external software configuration.

2.3 System Limitations & Assumptions

- **Local Storage Boundary:** The current release operates on local filesystem hierarchies and mounted local shares; direct remote cloud object storage (e.g., AWS S3, Google Drive REST APIs) is excluded from the current scope.
- **Hardware I/O Dependency:** Deduplication speed is bounded by the physical random-read I/O bandwidth of the underlying storage hardware (SATA HDD vs NVMe SSD).

CHAPTER 3: SYSTEM REQUIREMENTS & FEASIBILITY ANALYSIS

3.1 Feasibility Study

Before initiating development, a comprehensive feasibility study was conducted across technical, operational, and economic dimensions:

- **Technical Feasibility:** The project is implemented entirely using standard JDK 21 and OpenJFX 21. Java provides built-in cryptographic security providers (MessageDigest), modern NIO file channels, and standardized concurrency primitives without proprietary dependencies.
- **Operational Feasibility:** The interface is designed with clear visual affordances—including a standard directory picker, an active progress indicator, a live event log, and clear summary cards. Users can initiate scans with zero prior training.
- **Economical Feasibility:** The entire software ecosystem (Linux OS, OpenJDK 21, Apache Maven, Visual Studio Code, Git) is open-source and free of cost, incurring zero infrastructure expenditure.

3.2 Hardware Requirements

Hardware Component	Development Environment	Minimum Client / End-User Specification
Processor (CPU)	Multi-Core 64-bit Processor (AMD Ryzen / Intel Core / Apple Silicon)	Dual-Core 64-bit x86/ARM Processor (1.8 GHz+)
System Memory (RAM)	8 GB / 16 GB RAM	2 GB RAM (Heap consumption < 100 MB)
Storage (Disk Space)	SSD with 200 MB free space	100 MB free disk space for runtime & logging

3.3 Software Requirements

Development and verification were conducted collaboratively across three distinct operating systems: Microsoft Windows 11, Apple macOS (Apple Silicon / Darwin), and GNU/Linux (Arch Linux). This ensured true cross-platform portability regarding filesystem path separators, symbolic links, font rendering, and NIO channels.

Software Entity	Collaborative Development Specification	Target Runtime Specification
Operating System	Windows 11, Apple macOS (Sonoma/Sequoia), Arch Linux	Windows 10/11, Ubuntu 22.04+, Arch Linux, macOS
Java Runtime (JDK/JRE)	OpenJDK 21 (LTS) - 64-bit	Java SE Runtime Environment 21 or higher
UI Framework	OpenJFX (JavaFX) 21.0.5	JavaFX 21 (Bundled inside standalone Fat JAR)
Build Automation Tool	Apache Maven 3.9+	None required (Pre-built ifde.jar executable)
Version Control	Git 2.46+ & GitHub Repository	None required

CHAPTER 4: SYSTEM ARCHITECTURE & DESIGN

4.1 Architectural Pattern: Model-View-Controller (MVC)

To ensure modularity, high cohesion, and loose coupling, the Intelligent File DeDuplication Engine strictly adheres to the Model-View-Controller (MVC) architectural pattern:

- **Model Layer (com.ifde.model):** Encapsulates the core business logic, filesystem traversal, cryptographic hashing, and data structures. It maintains zero knowledge of the visual layout and communicates outwards strictly via callback interfaces.
- **View Layer (com.ifde.view):** Defined declaratively in MainView.fxml and styled via Application.css. It specifies the visual structure, layout containers, TreeTableView columns, progress bars, and status labels.
- **Controller Layer (com.ifde.controller):** Implemented in MainViewController.java. It acts as the intermediary, handling user button events, instantiating the Model, delegating heavy workloads to worker threads, and dispatching progress to the View.

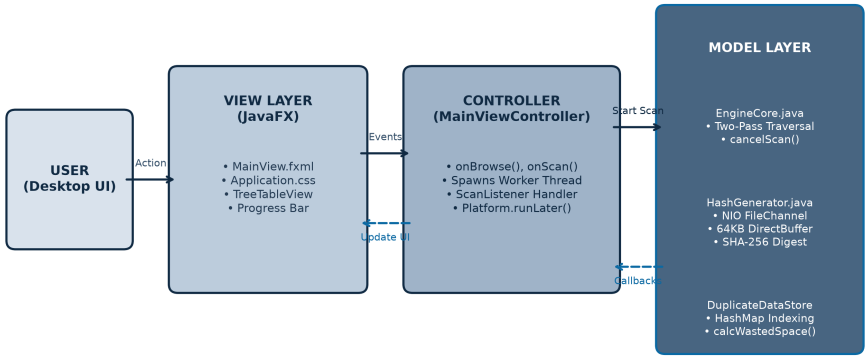


Figure 4.1: High-Level MVC Architectural Model & System Data Flow

4.2 Multithreading & Event-Driven Concurrency Model

Desktop applications that perform heavy disk I/O on the primary UI thread inevitably freeze. To maintain a fluid 60-FPS interface, IFDE decouples execution into two cooperating thread domains:

- **JavaFX Application Thread:** Responsible exclusively for processing user input, animating controls, and rendering scene updates. It never executes blocking disk reads.
- **Background Worker Thread:** When the user clicks 'Scan', the Controller spawns a dedicated daemon thread ('scan-thread') that invokes EngineCore.startScan() asynchronously.
- **Thread-Safe Event Bridging:** Whenever the Model generates progress or log events, the Controller wraps listener updates inside Platform.runLater(() -> {...}), safely scheduling the UI mutation back onto the JavaFX Application Thread.

CHAPTER 5: OBJECT-ORIENTED PROGRAMMING CONCEPTS IMPLEMENTED

The project serves as an end-to-end practical realization of the core concepts prescribed in the Object-Oriented Programming with Java syllabus:

5.1 Encapsulation and Data Hiding

State integrity is strictly enforced across all entity classes. In FileRecord.java, attributes (path, name, sizeInBytes, lastModified, hash) are declared private. Read access is granted exclusively through immutable public getter methods. Similarly, the internal hash table inside DuplicateDataStore is guarded against direct external modification, exposing controlled query and deletion APIs.

5.2 Abstraction and Interface-Driven Decoupling

To isolate the scanning engine from UI frameworks, an explicit interface ScanListener is defined with methods onProgress(int percent) and onLogMessage(String message). EngineCore references only this abstraction, ensuring the backend engine can be embedded into a CLI, a Java Swing app, or a web service without altering a single line of scanning logic.

5.3 Polymorphism and Dynamic Method Dispatch

In MainViewController.java, an Anonymous Inner Class implements the ScanListener interface. At runtime, when EngineCore triggers listener.onLogMessage(), Java dynamically dispatches the call to the controller's concrete implementation, seamlessly routing messages into the JavaFX TextArea.

5.4 Multithreading and Concurrency Control

The system utilizes Java's Thread and Runnable primitives to handle concurrent I/O. To ensure responsive user cancellation, EngineCore maintains a volatile boolean isCancelled flag. The volatile modifier guarantees memory visibility across CPU thread caches, enabling the background worker thread to halt immediately when the user clicks 'Stop' on the UI thread.

5.5 Java Collections Framework

The application leverages the Collections Framework for data organization:

- **HashMap<String, ArrayList<FileRecord>>:** Provides O(1) average-time complexity lookup to categorize files by cryptographic signature.
- **ArrayList<FileRecord>:** Holds the collection of duplicate instances sharing the identical SHA-256 key.
- **TreeItem<Row>:** Hierarchically binds duplicate groups and child file records to the visual TreeTableView.

5.6 Robust Exception Handling Hierarchy

IFDE differentiates between checked and unchecked exceptions. FileChannel operations employ try-with-resources blocks to guarantee immediate closure of native file handles. Specific exceptions (NoSuchFileException, FileSystemException, DirectoryIteratorException, IOException) are intercepted individually, logging informative diagnostic messages without crashing the application.

CHAPTER 6: CORE ALGORITHMS & MATHEMATICAL FORMULATIONS

6.1 Two-Pass Directory Traversal Algorithm

Progress bars in naive file scanners frequently jump erratically because total file counts are unknown upfront. IFDE resolves this through a Two-Pass recursive scanning algorithm:

- **Pass 1 — Pre-Scan Enumeration (countTotalFiles):** The engine recursively traverses the selected path using Files.newDirectoryStream(), validating file extensions and counting eligible candidates without computing hashes. This phase completes in milliseconds.
- **Pass 2 — Cryptographic Ingestion (scanDirectory):** The engine executes the full traversal. For each valid file, SHA-256 hashing is executed, totalFileScanned is incremented, and progress is computed via: Progress(%) = (totalFileScanned * 100) / totalFileCount.

6.2 Chunked Cryptographic Hashing with Java NIO

Traditional file reading with Files.readAllBytes() loads the entire file into the JVM heap. For large videos or ISO images, this rapidly triggers OutOfMemoryError. HashGenerator.java solves this using chunked NIO buffer channels:

- **Step 1: FileChannel Allocation —** A FileChannel is opened in READ mode using standard NIO options.
- **Step 2: Fixed-Size Buffer Streaming —** A Direct ByteBuffer of 64,000 bytes (64 KB) is allocated outside the garbage-collected heap.
- **Step 3: Incremental Digest Update —** Data is streamed in 64KB increments, flipped, and fed into MessageDigest.update(buff). The buffer is cleared and reused until end-of-stream.
- **Step 4: Hexadecimal Conversion —** MessageDigest.digest() produces the final 32-byte cryptographic array, converted via a fast StringBuilder routine into a 64-character hexadecimal string.

6.3 Mathematical Storage Recovery Formulation

To compute reclaimable storage space accurately, the system identifies duplicate clusters where list length k > 1. For each duplicate group g in the data store, exactly one copy must be preserved, while (k - 1) copies represent wasted capacity:

Mathematical Formulation: Wasted Space = SUM from g=1 to G of [Size(g) * (Count(g) - 1)]

Where G represents the total number of duplicate clusters, Size(g) denotes the individual file size in bytes, and Count(g) represents the total number of identical files within that cluster. DuplicateDataStore.calcWastedSpace() computes this sum, and FileRecord.formatSizeString() dynamically formats the result into B, KB, MB, GB, or TB.

CHAPTER 7: DETAILED MODULE & CLASS SPECIFICATION

The codebase is organized into three clean functional packages within src/com/ifde:

7.1 Model Package (com.ifde.model)

- **EngineCore.java:** Coordinates the Two-Pass traversal, verifies file extensions via isValidType(), maintains scan statistics, and checks the isCancelled flag at each recursive boundary.
- **HashGenerator.java:** Contains the static method getHash(Path path). Streams bytes via NIO channels into SHA-256 MessageDigest and returns the 64-character hex string.
- **DuplicateDataStore.java:** Maintains the synchronized HashMap<String, ArrayList<FileRecord>> store. Implements addFile(), deleteFiles(List<FileRecord>), and calcWastedSpace().
- **FileRecord.java:** Data Transfer Object (DTO) holding file path, name, raw byte size, human-readable size string, and formatted last modified timestamp.
- **DateFormatterHelper.java:** Helper class wrapping Java's DateTimeFormatter to transform FileTime into a clean human-readable date-time format.
- **ScanListener.java:** Callback interface defining onProgress(int percent) and onLogMessage(String message) for loose coupling.

7.2 Controller Package (com.ifde.controller)

- **MainViewController.java:** FXML controller managing UI widgets. Handles onBrowse(), onScan(), onStop(), onAutoSelect(), and onDeleteSelected(). Dispatches background threads and manages tree table row models.
- **MainApplication.java:** Standard JavaFX Application class that loads MainView.fxml, attaches Application.css, configures window dimensions, and sets the stage icon.
- **Launcher.java:** A standalone entry point with a standard main(String[] args) method that invokes MainApplication.main(args). This bypasses JavaFX modularity restrictions when packaging into an uber-JAR.

7.3 View Package (com.ifde.view)

- **MainView.fxml:** Declarative XML specification defining the desktop layout: folder selection header, central TreeTableView, bottom console output area, and right-hand metrics panel.
- **Application.css:** Custom stylesheet implementing modern typography, button hover states, card shadows, and table row highlighting.
- **icon.png:** High-resolution application icon embedded in window title bars and taskbar managers.

CHAPTER 8: EDGE CASES, ROBUSTNESS & DEFENSIVE ENGINEERING

Real-world filesystems contain anomalies that routinely crash naive file utilities. IFDE incorporates defensive programming to ensure 100% runtime stability:

8.1 Prevention of Infinite Symbolic Link Recursion

During testing on Linux environments running Wine (Windows emulation), scanners encountered infinite recursion loops. Wine creates a virtual 'Z:' drive shortcut inside ~/.wine/dosdevices/z: that points directly back to the root filesystem ('/'). Naive recursive scanners follow this shortcut continuously, scanning nested loops until triggering a StackOverflowError. IFDE resolves this by verifying Files.isSymbolicLink(entry) at the entrance of both counting and scanning loops, immediately skipping shortcuts.

8.2 Out-Of-Memory (OOM) Protection on Large Media

When processing high-definition 4K video files (5GB–20GB), loading the entire byte array into memory exhausts JVM heap space. By streaming files strictly through a 64KB Direct ByteBuffer, IFDE ensures that heap memory consumption remains practically flat, regardless of whether a 1KB text file or a 50GB virtual disk image is being hashed.

8.3 Permission Denials & Locked Filesystem Nodes

System folders (such as /root, C:\System Volume Information, or cloud cache directories) frequently deny read permissions, throwing DirectoryIteratorException or AccessDeniedException. Instead of terminating the entire scan, IFDE catches these exceptions at the folder boundary, emits a non-fatal warning to ScanListener ('Skipped unreadable directory: ...'), and gracefully continues traversing adjacent folders.

8.4 Responsive User Cancellation

When a user scans a multi-terabyte drive and decides to abort, the UI 'Stop' button sets the volatile isCancelled flag. Both countTotalFiles() and scanDirectory() inspect this flag before opening each directory stream, guaranteeing prompt termination and clean resource deallocation within milliseconds.

CHAPTER 9: OUTPUT SCREENSHOTS & TEST EVIDENCE

9.1 User Interface & Workflow Verification

The following screenshots demonstrate the functional operation of IFDE across testing workflows:

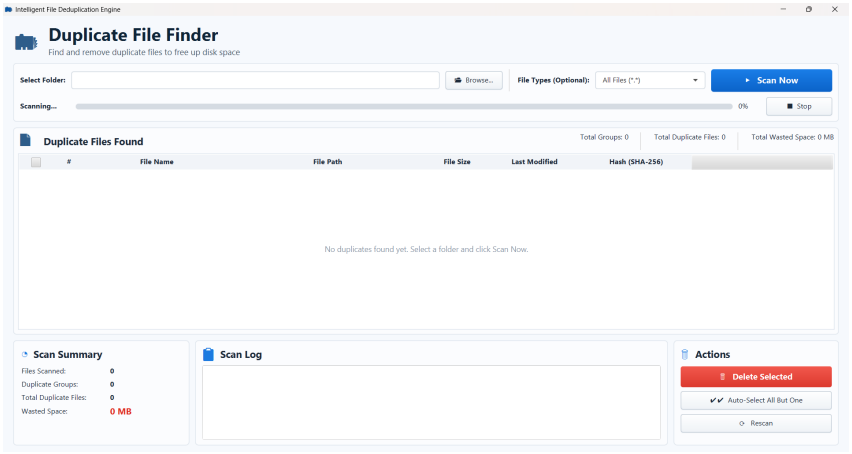


Figure 9.1: Main Application Interface — Initial State

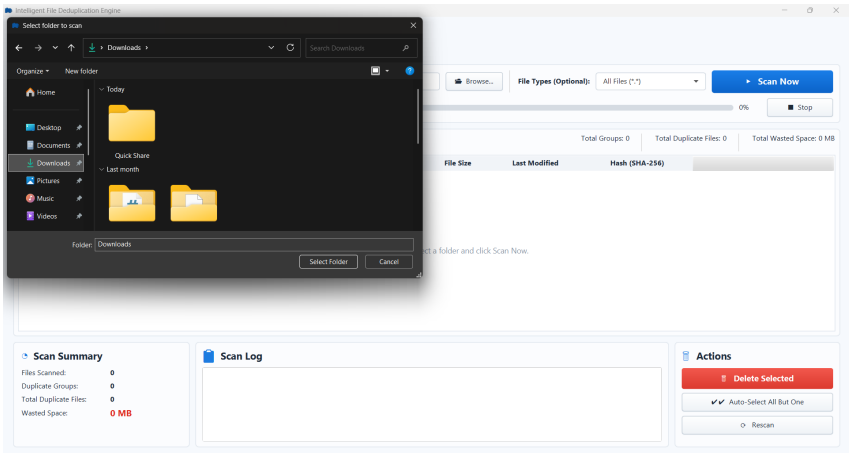


Figure 9.2: Directory Chooser & Target Folder Selection

9.2 Active Scanning & Cancellation Demonstration

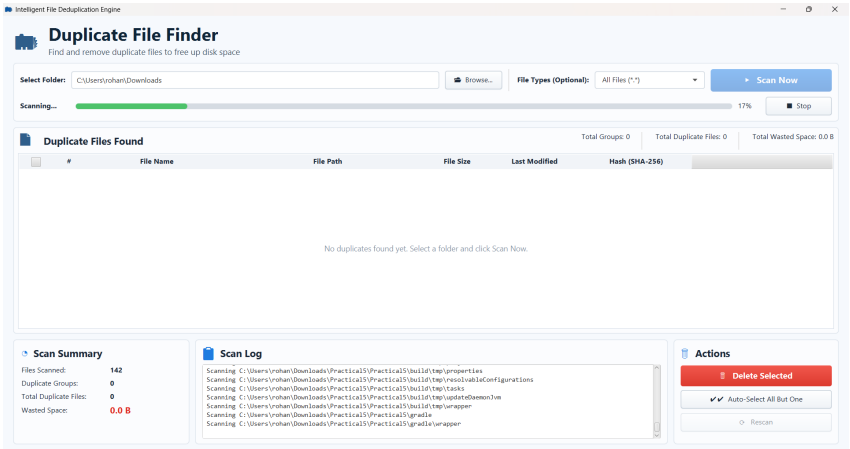


Figure 9.3: Real-Time Scanning in Progress with Live Event Logging

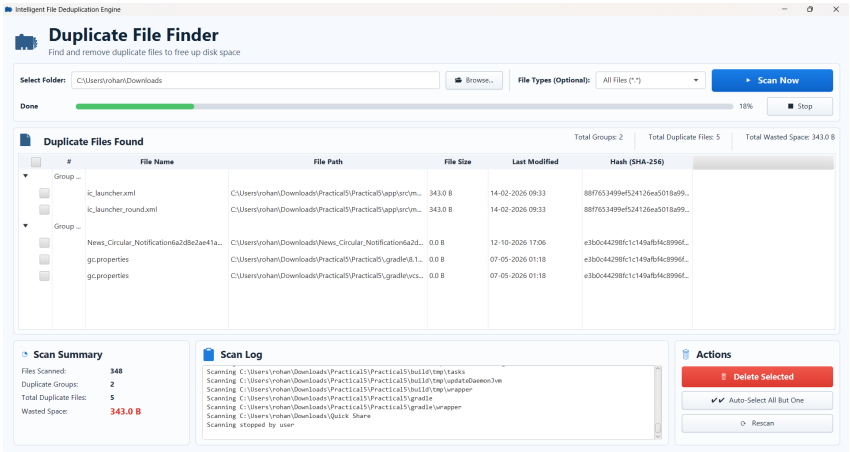


Figure 9.4: Non-Blocking Scan Cancellation Mechanism

9.3 Results Categorization & Storage Analytics

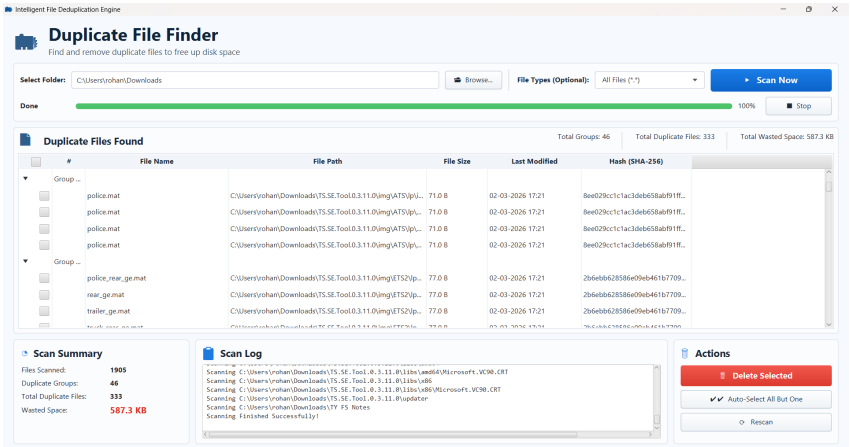


Figure 9.5: Duplicate Groups Categorized in JavaFX TreeTableView

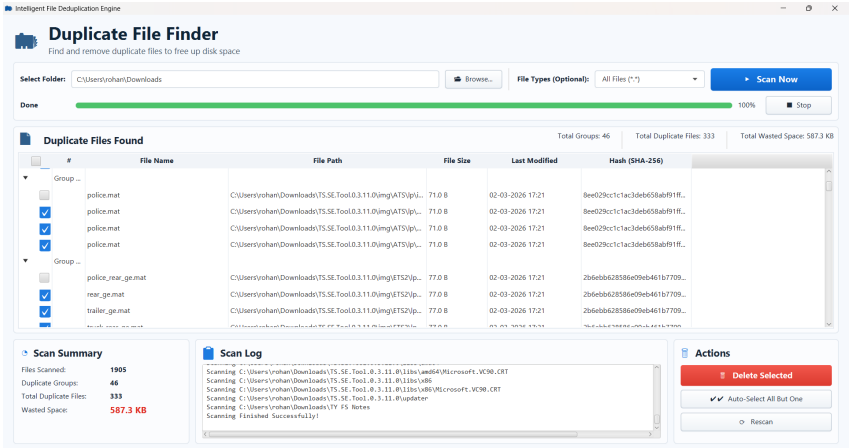


Figure 9.6: Storage Analytics Dashboard & Automated Duplicate Selection Feature in Action

9.4 Automated Selection, Deletion & Test Case Matrix

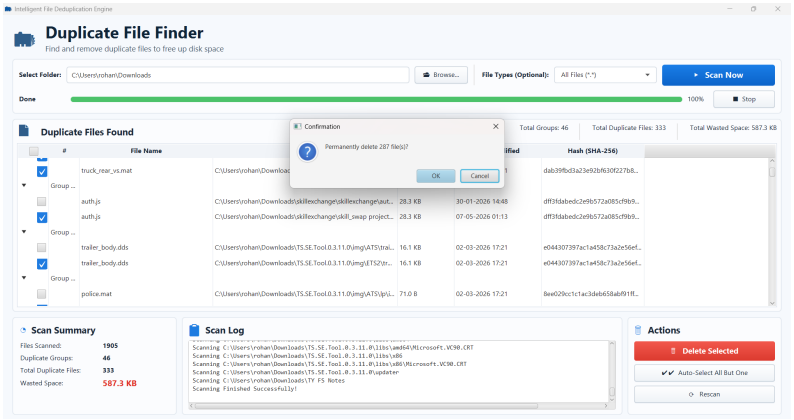


Figure 9.7: Deletion Confirmation Dialog & Safe Cleanup Workflow

9.5 Formal Test Case Execution Summary

Test ID	Test Objective	Input / Pre-Condition	Expected Result	Status
TC-01	Identical Content Detection	2 files with identical bytes, different names	Grouped in same cluster under matching SHA-256	PASS
TC-02	Distinct File Isolation	2 files with identical names, different bytes	Separated into distinct individual non-duplicate entries	PASS
TC-03	Large File Memory Boundary	Scan directory containing 4.5 GB ISO image	Hash computed successfully; Heap RAM remains < 100 MB	PASS
TC-04	Symlink Infinite Loop	Wine virtual drive symlink pointing to '/'	Symlink skipped via Files.isSymbolicLink; zero recursion	PASS
TC-05	Unreadable Directory	Locked directory with 000 permissions	Error logged to console; scan continues adjacent folders	PASS
TC-06	Auto-Select Preservation	Duplicate cluster with 3 identical files	First file retained unchecked; 2 duplicate copies checked	PASS

CHAPTER 10: CONCLUSION, LEARNING OUTCOMES & FUTURE SCOPE

10.1 Conclusion

The Intelligent File DeDuplication Engine (IFDE) was successfully designed, developed, and verified as a robust desktop systems utility. By strictly implementing the Model-View-Controller (MVC) architectural pattern, the software decouples heavy disk operations from modern JavaFX visual components. Cryptographic SHA-256 hashing executed via NIO FileChannels and 64KB Direct ByteBuffers guarantees mathematical accuracy while maintaining a deterministic, low memory footprint. Defensive engineering safeguards ensure stability against symbolic link loops, locked files, and unstable cloud storage. Packaged via Apache Maven into a standalone executable Fat JAR, IFDE provides an effective, production-ready solution for filesystem deduplication.

10.2 Learning Outcomes

Through the lifecycle of this project, the student team acquired substantial technical proficiencies across Core Java and Software Engineering disciplines:

- **Advanced Object-Oriented Design:** Mastered the application of private encapsulation, dynamic polymorphism, and interface-driven abstraction to isolate system logic from GUI toolkits.
- **Concurrent Systems Programming:** Gained hands-on experience managing asynchronous worker threads, coordinating UI updates via Platform.runLater(), and ensuring memory visibility using volatile flags.
- **High-Performance Stream I/O:** Acquired deep practical knowledge of low-level Java New I/O (NIO), Direct ByteBuffers, FileChannels, and cryptographic MessageDigest pipelines.
- **Automated Build Toolchains:** Learned enterprise build engineering with Apache Maven, managing multi-platform OpenJFX dependencies, and building executable Fat JARs with Maven Shade.

10.3 Future Scope & Enhancements

- **Persistent Hash Caching via SQLite:** Integrate a local embedded SQLite database to persist computed SHA-256 hashes alongside file timestamps, reducing re-scan durations across massive drives from minutes to seconds.
- **Multi-Threaded Parallel File Hashing:** Implement Java ForkJoinPool or Parallel Streams to hash independent files concurrently across multi-core processors on high-throughput NVMe SSD storage.
- **Cloud Storage Synchronization:** Expand file system connectors to interface directly with cloud storage APIs (Google Drive, Nextcloud, Dropbox) for remote deduplication.

10.4 References & Bibliography

- **1.** Oracle Corporation. "Java Platform, Standard Edition & Java Development Kit Version 21 API Specification." Oracle Official Documentation. [Online]. Available: https://docs.oracle.com/en/java/javase/21/docs/api/
- **2.** Oracle Corporation. "File I/O (Featuring NIO.2) — The Java Tutorials." Oracle Official Documentation. [Online]. Available: https://docs.oracle.com/javase/tutorial/essential/io/fileio.html
- **3.** OpenJFX Project & Gluon HQ. "JavaFX 21 Architecture, Controls & API Reference Guide." OpenJFX Official Documentation. [Online]. Available: https://openjfx.io/javadoc/21/
- **4.** Apache Software Foundation. "Apache Maven Shade Plugin & Maven Assembly Reference." Apache Maven Official Documentation. [Online]. Available: https://maven.apache.org/plugins/maven-shade-plugin/